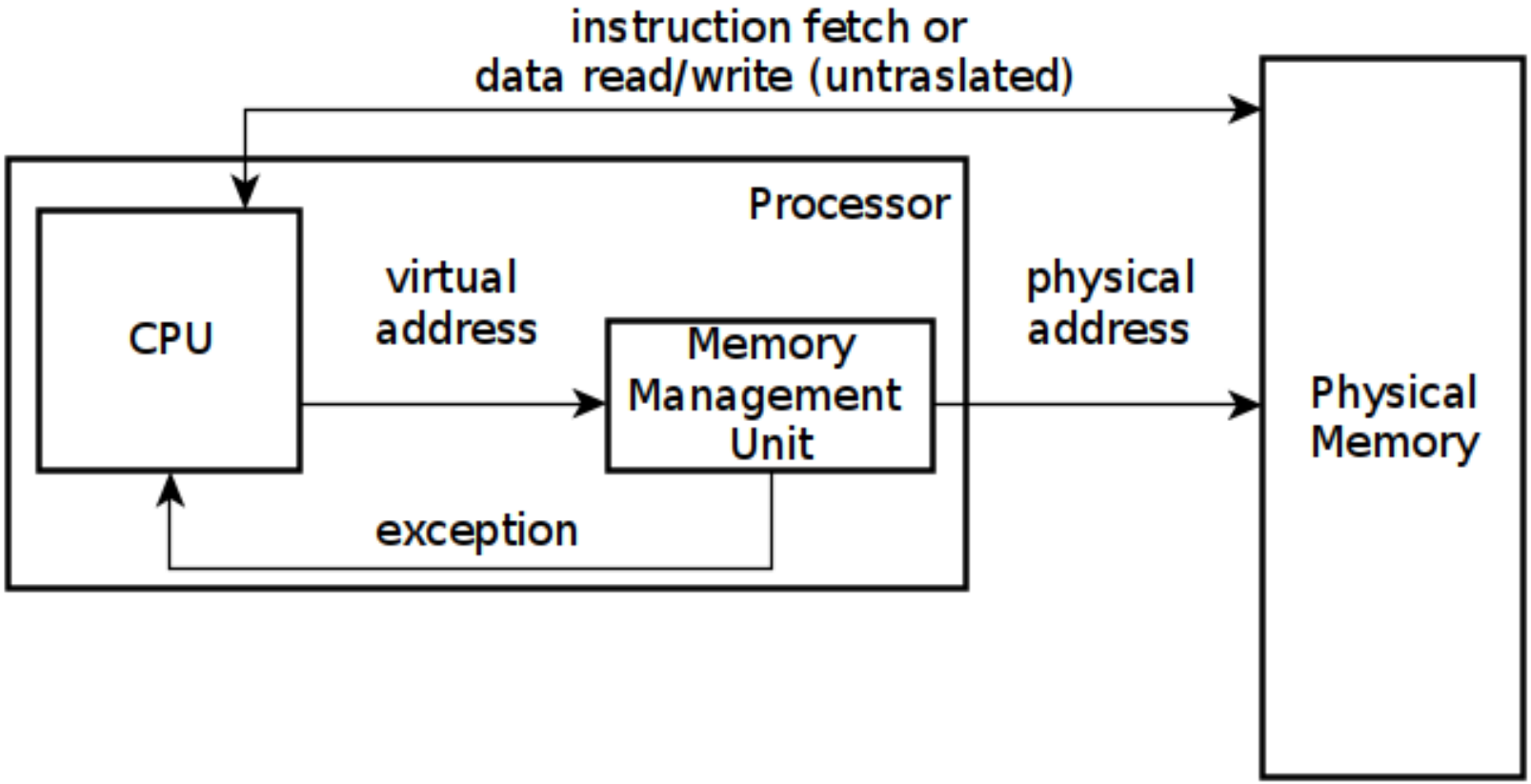
Address Translation

Main Points

- Address Translation Concept
  - How do we convert a virtual address to a physical address?
- Flexible Address Translation
  - Base and bound
  - Segmentation
  - Paging
  - Multilevel translation
- Efficient Address Translation
  - Translation Lookaside Buffers (TLB)
  - Virtually and Physically Addressed Caches

Address Translation Concept



Address Translation Goals

- Memory protection
- Memory sharing
- Flexible memory placement
- Sparse addresses
- Runtime lookup efficiency
- Compact translation tables
- Portability

Address Translation

- What can you do if you can (selectively) gain control whenever a program reads or writes a particular memory location?
  - With hardware support
  - With compiler-level support
- Memory management is one of the most complex parts of the OS
  - Serves many different purposes
  - … implements a virtual memory

Address Translation Uses

- Process isolation
  - Keep a process from touching anyone else's memory, or the kernel's
- Efficient interprocess communication
  - Shared regions of memory between processes
- Shared code segments
  - E.g., common libraries used by many different programs
- Program initialization
  - Start running a program before it is entirely in memory
- Dynamic memory allocation
  - Allocate and initialize stack/heap pages on demand

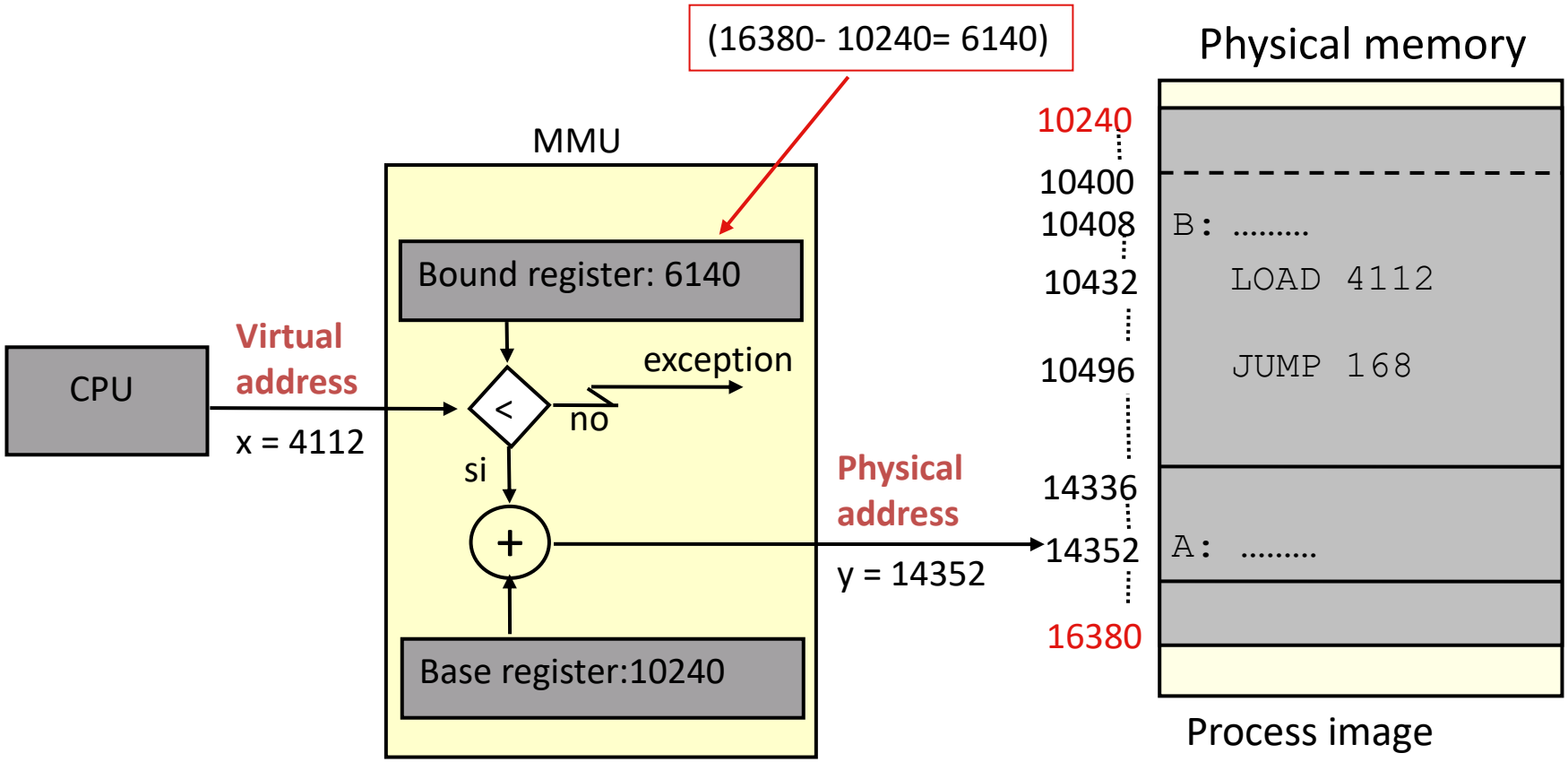
Address Translation (more)

- Cache management
  - Page coloring
- Program debugging
  - Data breakpoints when address is accessed
- Zero-copy I/O
  - Directly from I/O device into/out of user memory
- Memory mapped files
  - Access file data using load/store instructions
- Demand-paged virtual memory
  - Illusion of near-infinite memory, backed by disk or memory on other machines

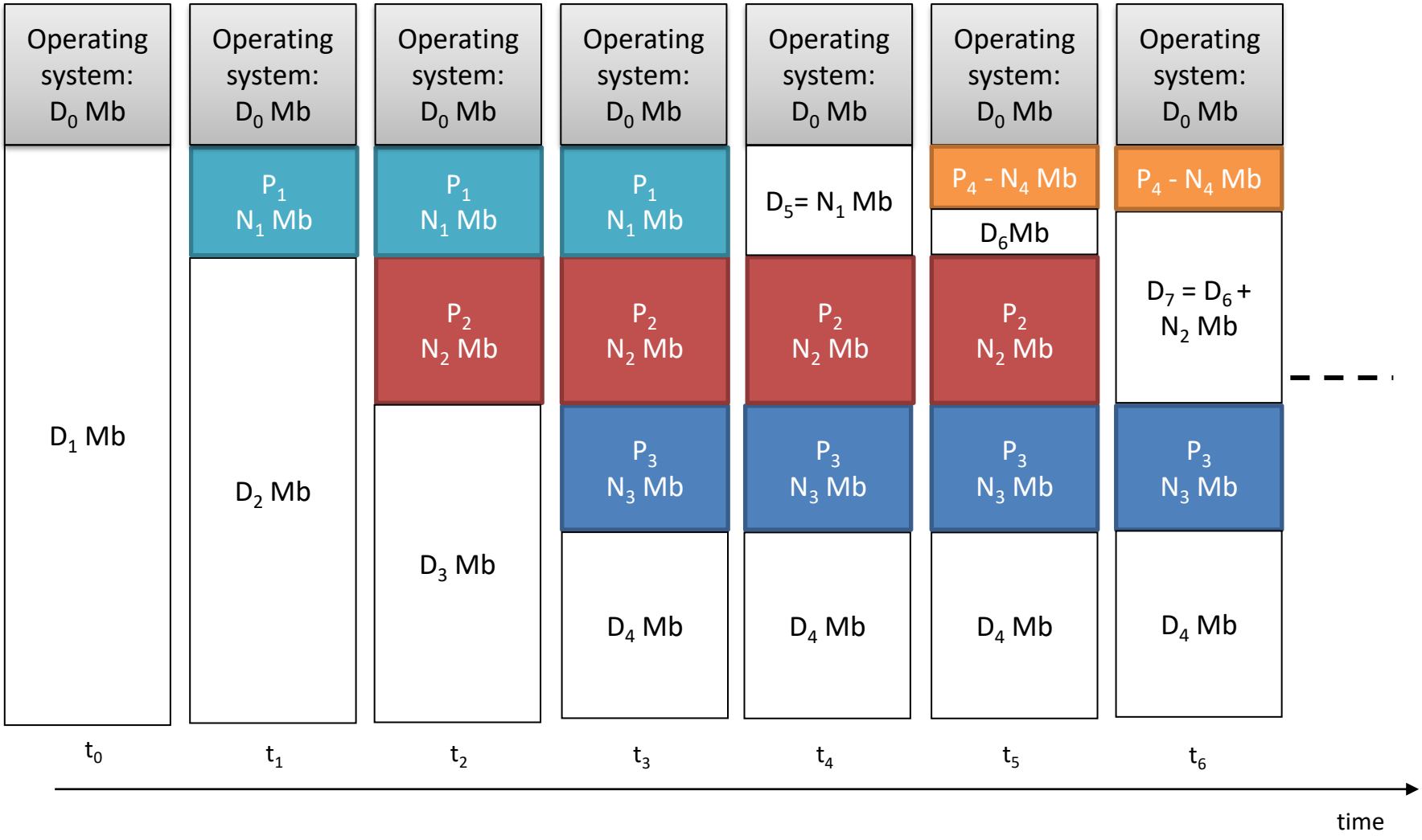
Address Translation (even more)

- Checkpointing/restart
  - Transparently save a copy of a process, without stopping the program while the save happens
- Persistent data structures
  - Implement data structures that can survive system reboots
- Process migration
  - Transparently move processes between machines
- Information flow control
  - Track what data is being shared externally
- Distributed shared memory
  - Illusion of memory that is shared between machines

Virtual Base and Bounds



Variable partitions



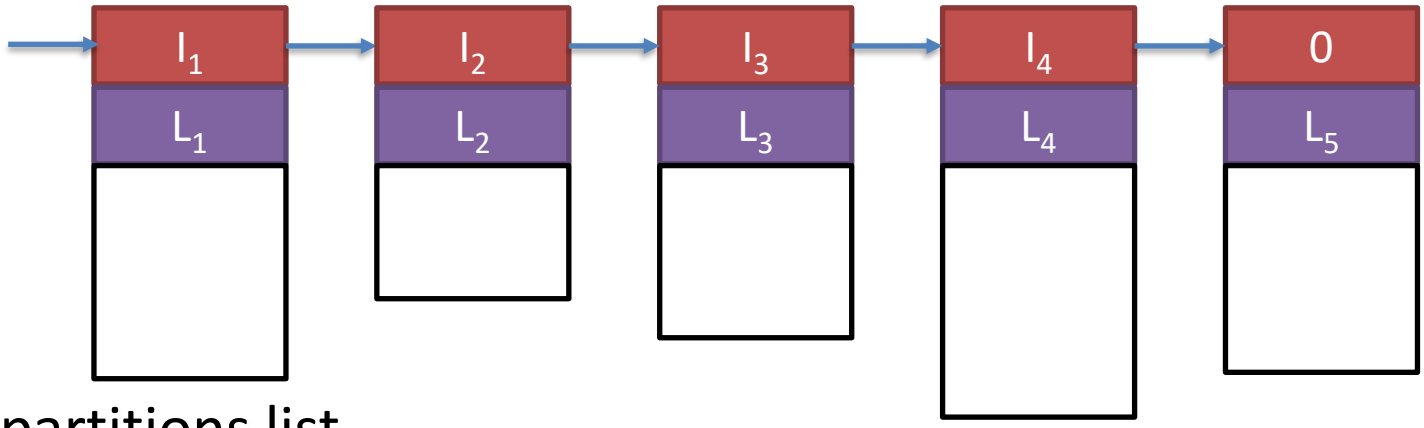
Allocation of a new partition

- **First-fit**

  among all free partitions take the first one that is large enough to allocate the new partition

- **Best-fit**

  among all free partitions take the smallest one that is large enough to allocate the new partition

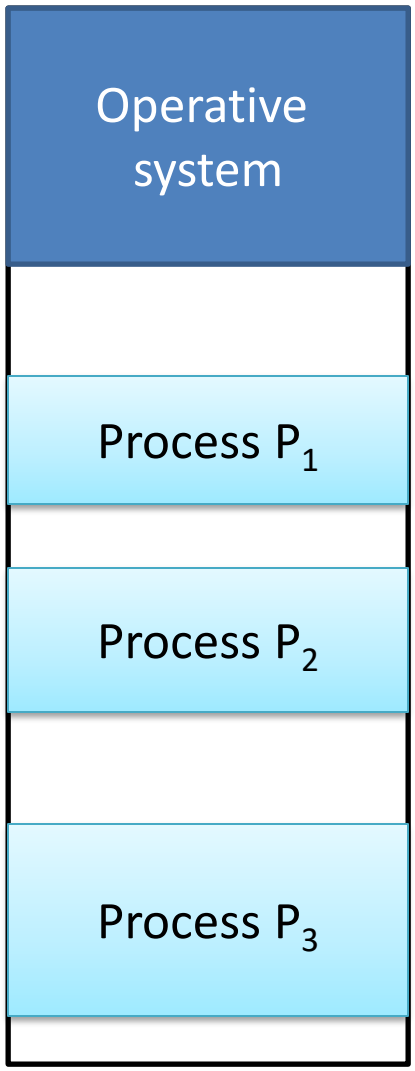


Free partitions list

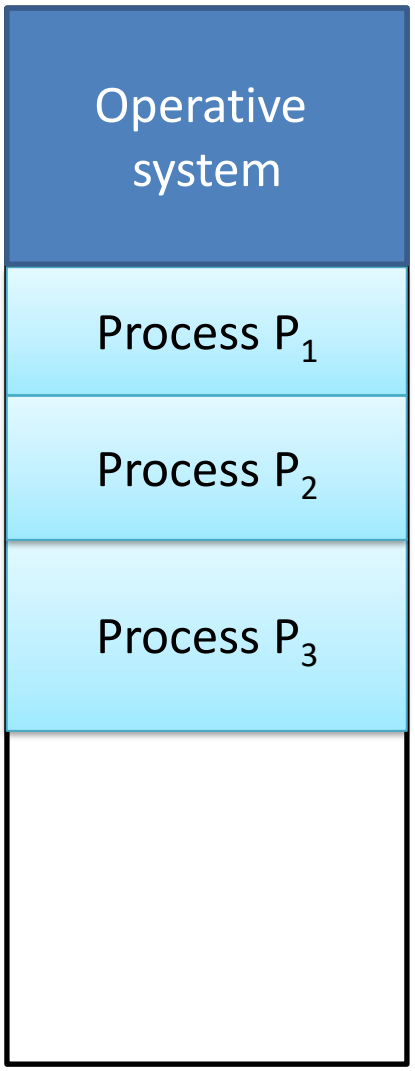
Fragmentation

- **Internal fragmentation**
  - memory allocated to a partition but not used/not necessary to the process.
  - Happens if fixed partitions are used.
- **External fragmentation**
  - the free memory partitions are too small to be used for other memory allocations.
  - Even if the overall free memory might be sufficient…
  - Occurs with variable partitions.

Memory de-fragmentation



Fragmented memory



De-fragmented memory

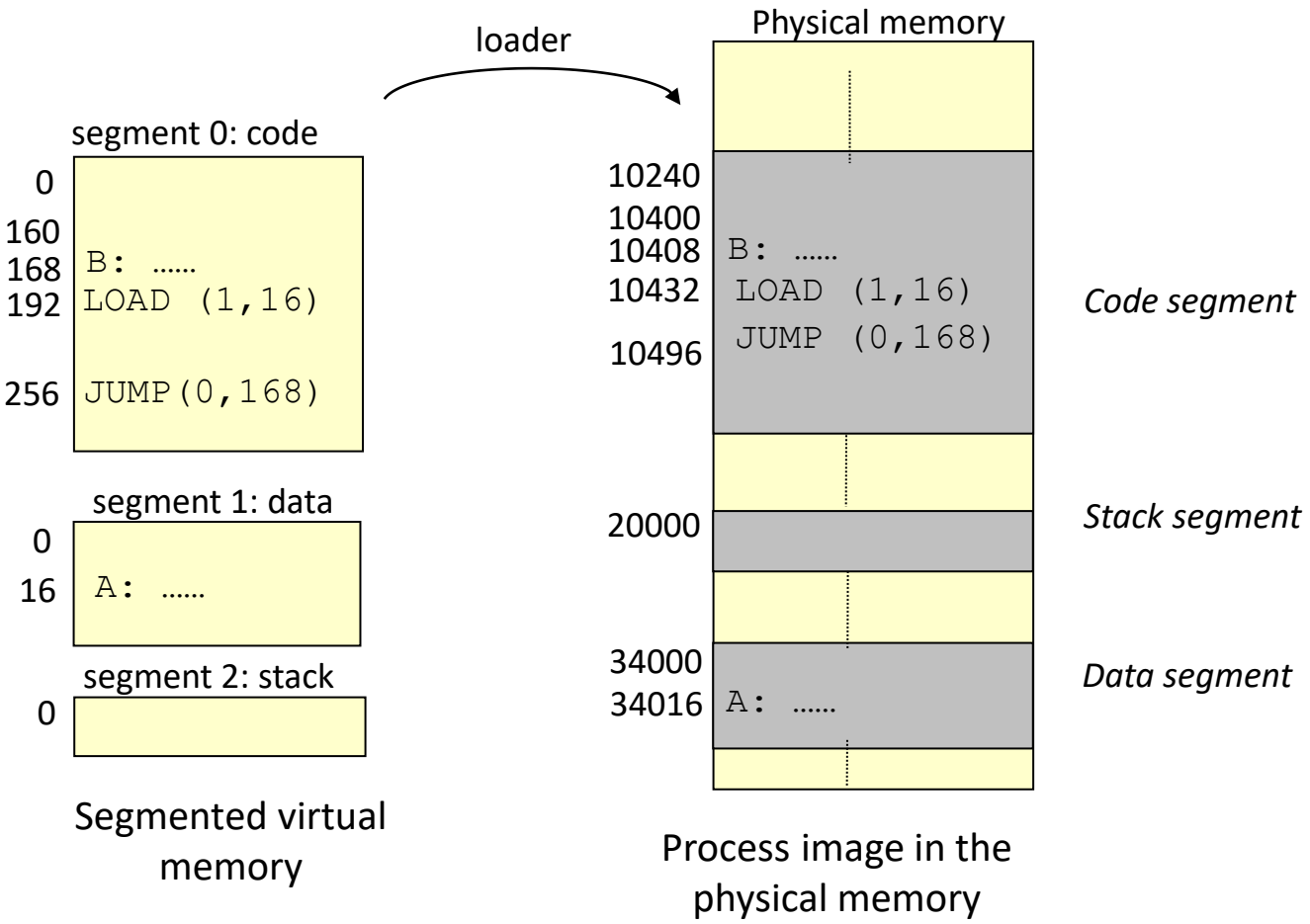
Virtual Base and Bounds

- Pros?
  - Simple
  - Fast (2 registers, adder, comparator)
  - Can relocate in physical memory without changing process
- Cons?
  - Can’t keep program from accidentally overwriting its own code
  - Can’t share code/data with other processes
  - Can’t grow stack/heap as needed

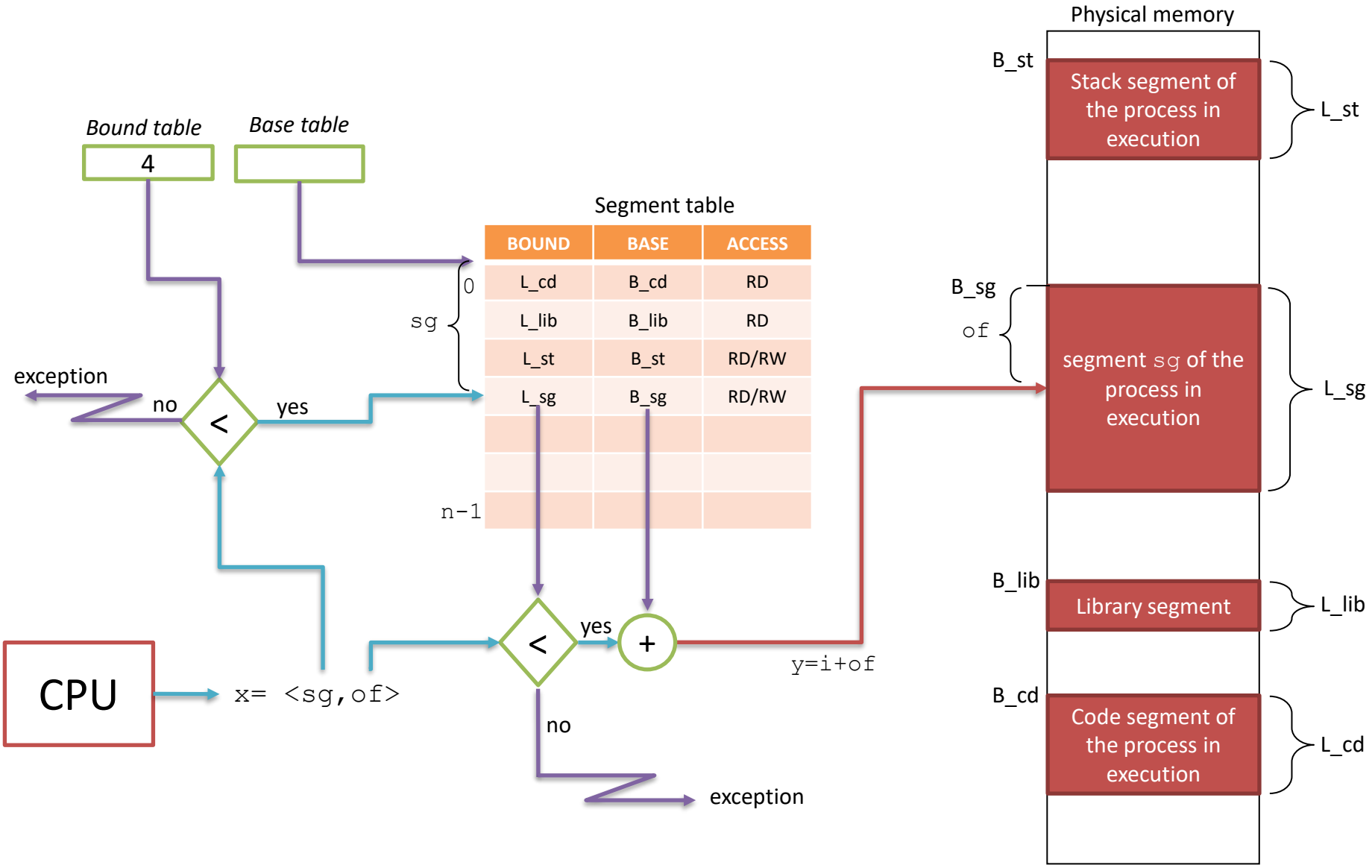
Segmentation

- Segment is a contiguous region of memory
  - Virtual or (for now) physical memory
- Each process has a segment table (in hardware)
  - Segment table = array of segment entries
  - Entry in table = segment (i.e., a base and bound pair)
- Segment can be located anywhere in physical memory
  - Start
  - Length
  - Access permission
- Processes can share segments
  - Same start, length, same/different access permissions

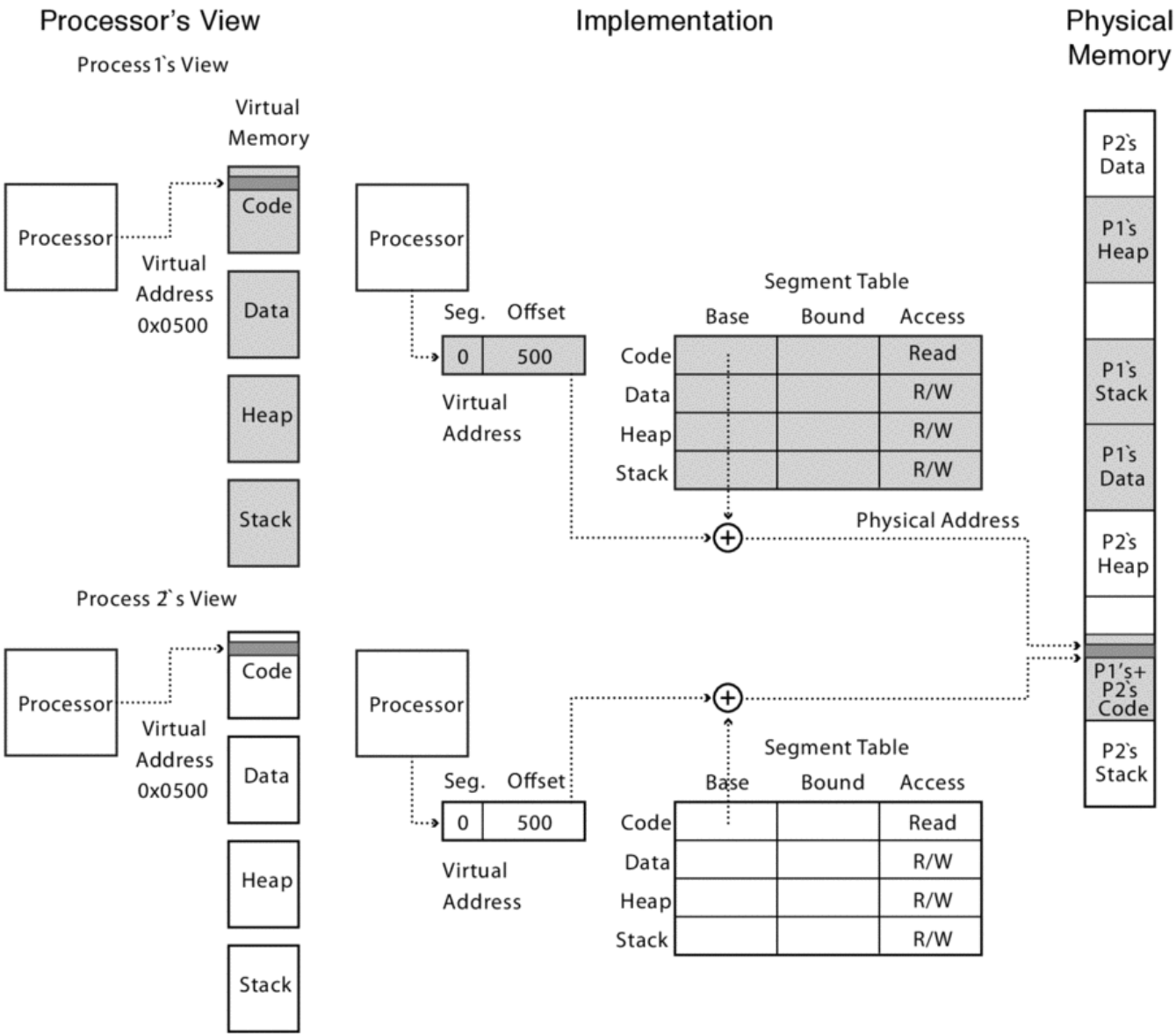
Segmentation



Segmentation: address translation



Segments sharing



UNIX fork and Copy on Write

- UNIX fork
  - Makes a complete copy of a process
- Segments allow a more efficient implementation
  - Copy segment table into child
  - Mark parent and child segments read-only
  - Start child process; return to parent
  - If child or parent writes to a segment, will trap into kernel
    - make a copy of the segment and resume

Zero-on-Reference

- How much physical memory do we need to allocate for the stack or heap?
  - Zero bytes!
- When program touches the heap
  - Segmentation fault into OS kernel
  - Kernel allocates some memory
    - How much?
  - Zeros the memory
    - avoid accidentally leaking information!
  - Restart process

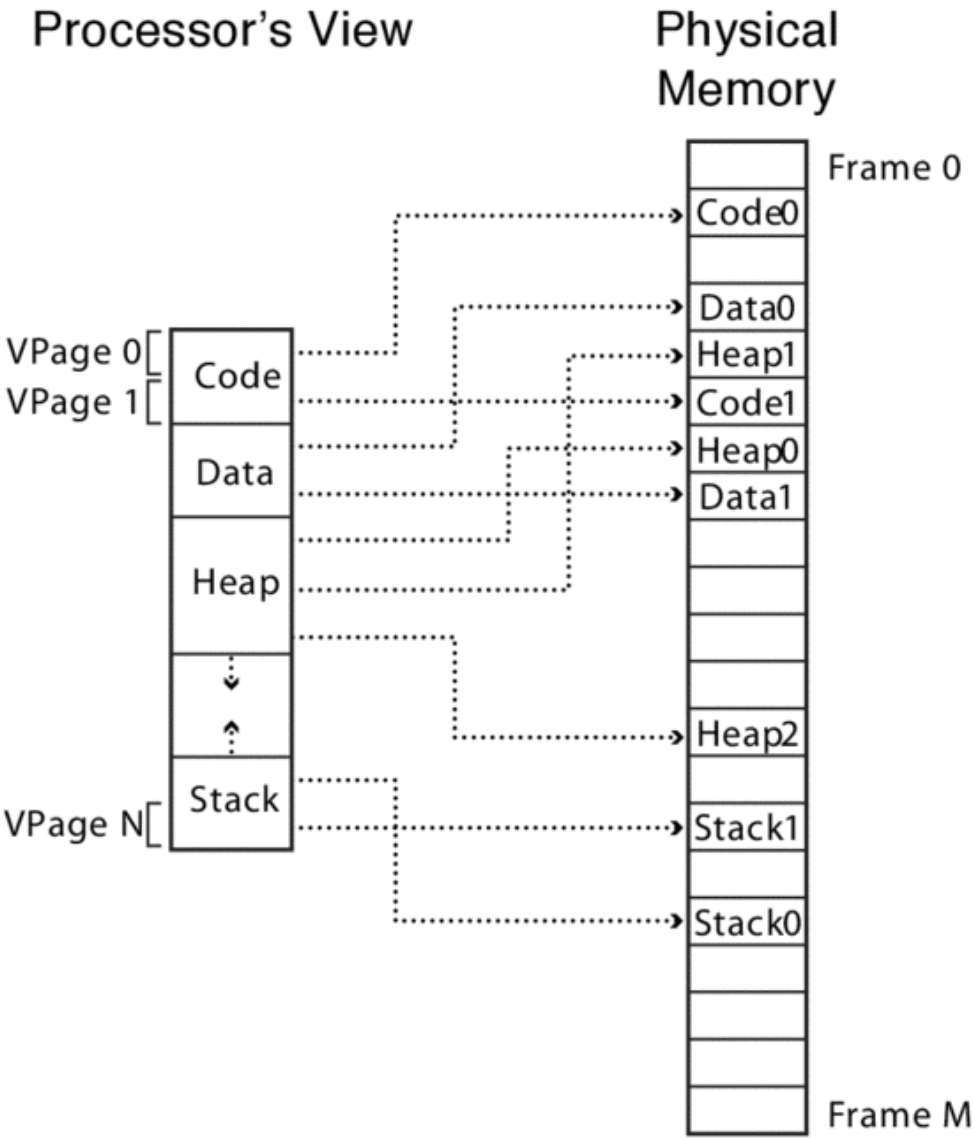
Segmentation

- Pros?
  - Can share code/data segments between processes
  - Can protect code segment from being overwritten
  - Can transparently grow stack/heap as needed
  - Can detect if need to copy-on-write
- Cons?
  - Complex memory management (mainly allocation)
    - Need to find chunk of a particular size
  - May need to rearrange memory from time to time to make room for new segment or growing segment
    - External fragmentation: wasted space between chunks

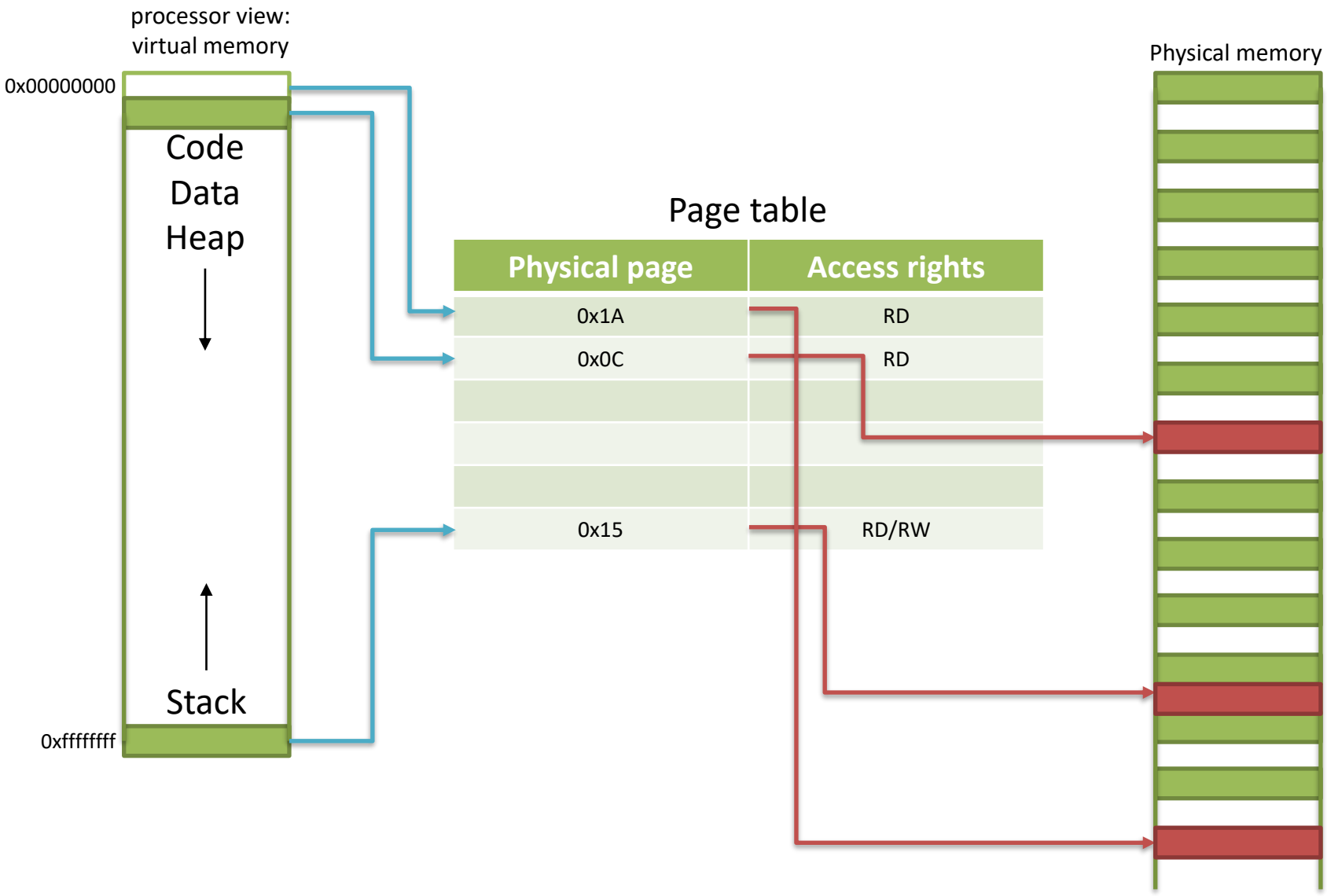
Paged Translation

- Manage memory in *fixed size units* (*page frames*)
- Finding a free page is easy
  - Bitmap allocation: 0011111100000001100
  - Each bit represents one physical page frame
- Each process has its own page table
  - **Page table stored in physical memory**. Why?
  - We need two registers
    - pointer to page table start
    - page table length

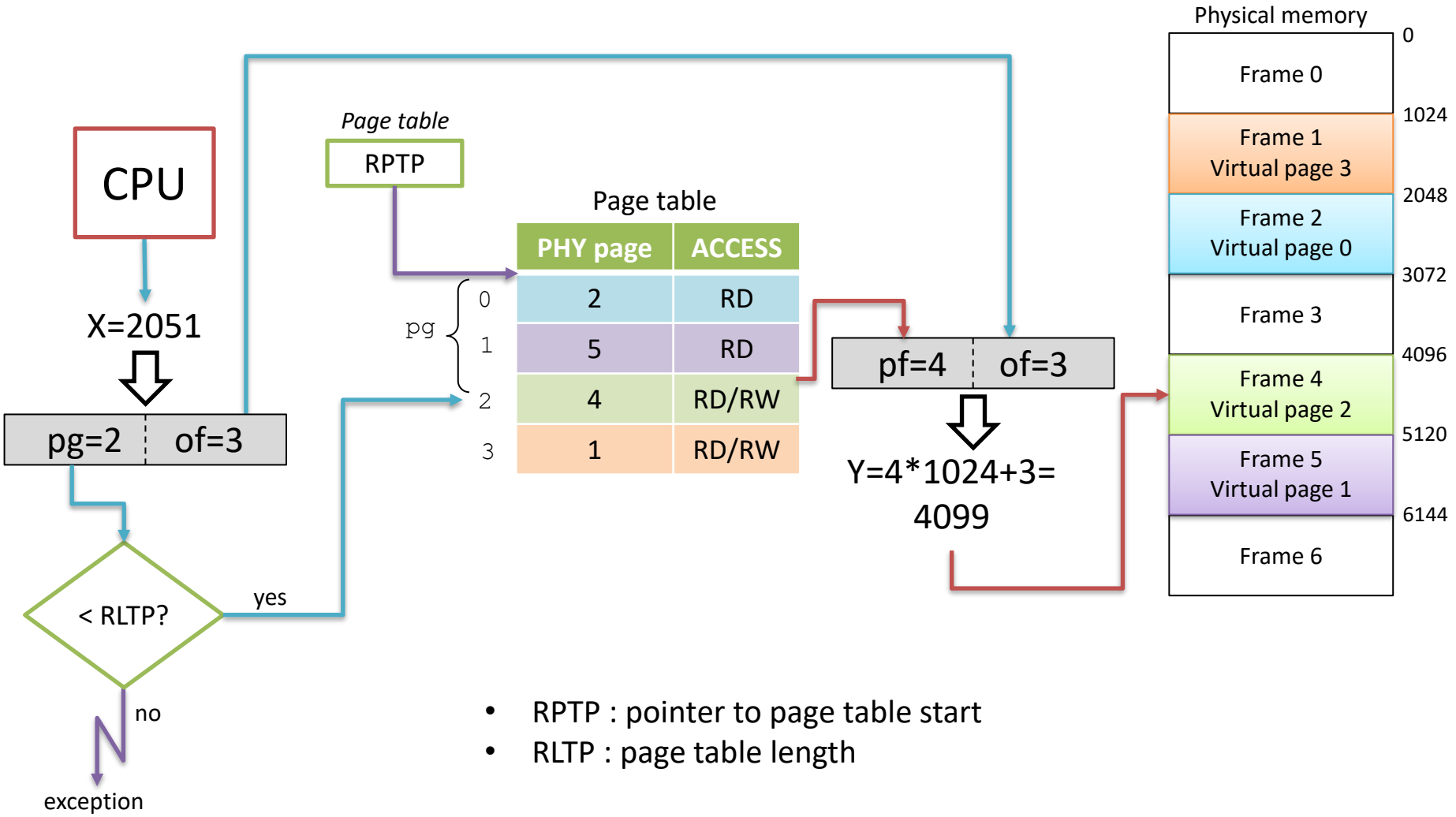
Virtual and physical spaces



Virtual and physical spaces



Paged translation



- RPTP : pointer to page table start
- RLTP : page table length

Paging Questions

- What must be saved/restored on a process context switch?
  - Pointer to page table/size of page table
  - Page table itself is in main memory
- What if page size is very small?
  - Huge page table
- What if page size is very large?
  - Internal fragmentation: if we don’t need all the space inside a fixed size chunk

Page Sharing and Copy on Write

- Can we share memory between processes?
  - Set entries in both page tables to point to the same page frames
  - Need ***core map*** of page frames, a data structure to track which processes are pointing to which page frames
- UNIX fork with Copy on Write (CoW) at page granularity
  - Copy page table entries to new process
  - Mark all pages as read-only
  - Trap into kernel on write (in child or parent)
  - Copy page and resume execution

Paging and Fast Program Start

- Can I start running a program before its code is in physical memory?
  - Set all page table entries to invalid
  - When a page is referenced for first time
    - Trap to OS kernel
    - OS kernel brings in page
    - Resumes execution
  - Remaining pages can be transferred in the background while program is running

Sparse Address Spaces

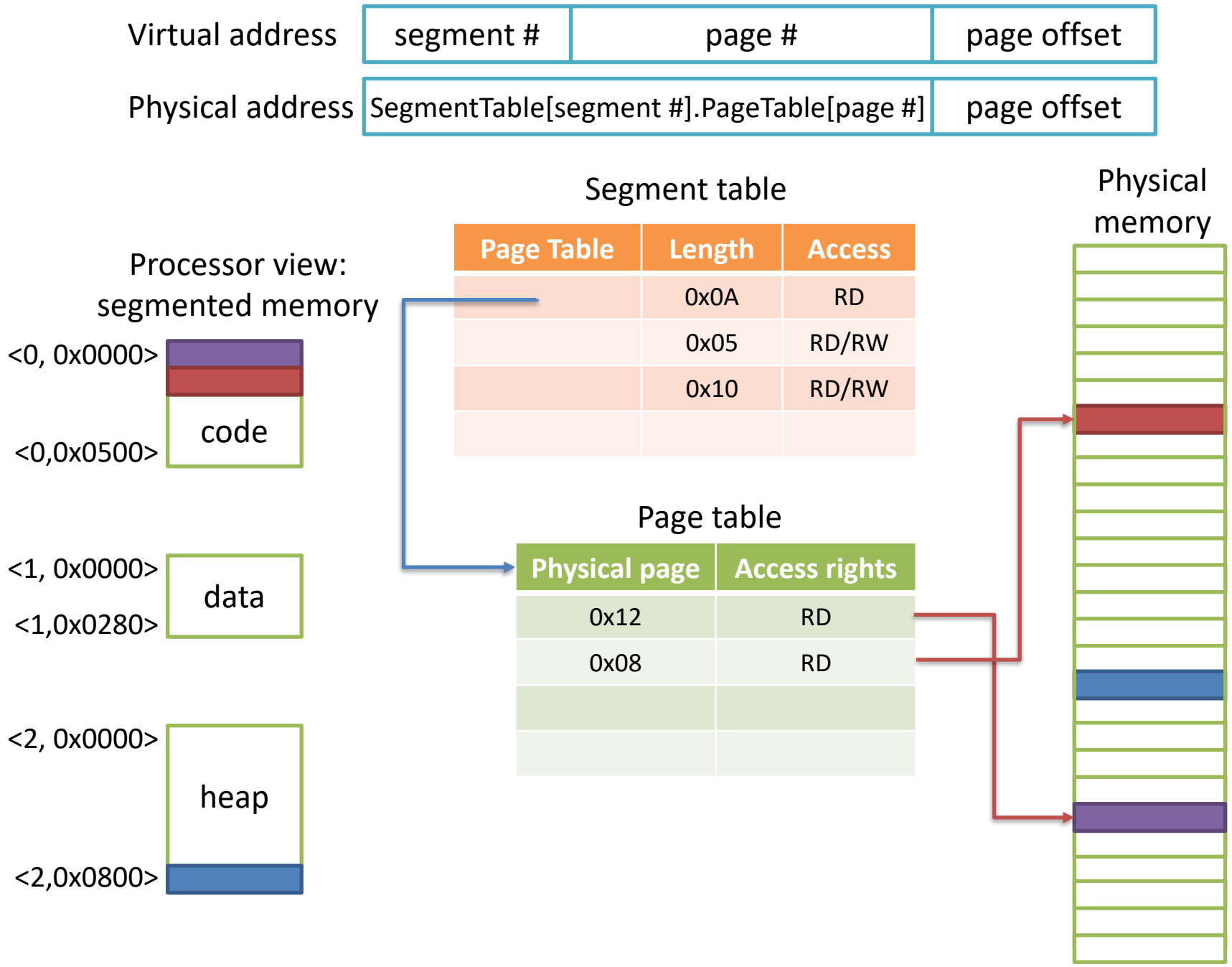
- Might want many separate segments
  - Per-processor heaps
  - Per-thread stacks
  - Memory-mapped files
  - Dynamically linked libraries
- What if virtual address space is sparse?
  - On 32-bit UNIX, code starts at 0
  - Stack starts at 2^32-1
  - 4KB pages => 1M page table entries
  - 64-bits => ~4 quadrillion page table entries

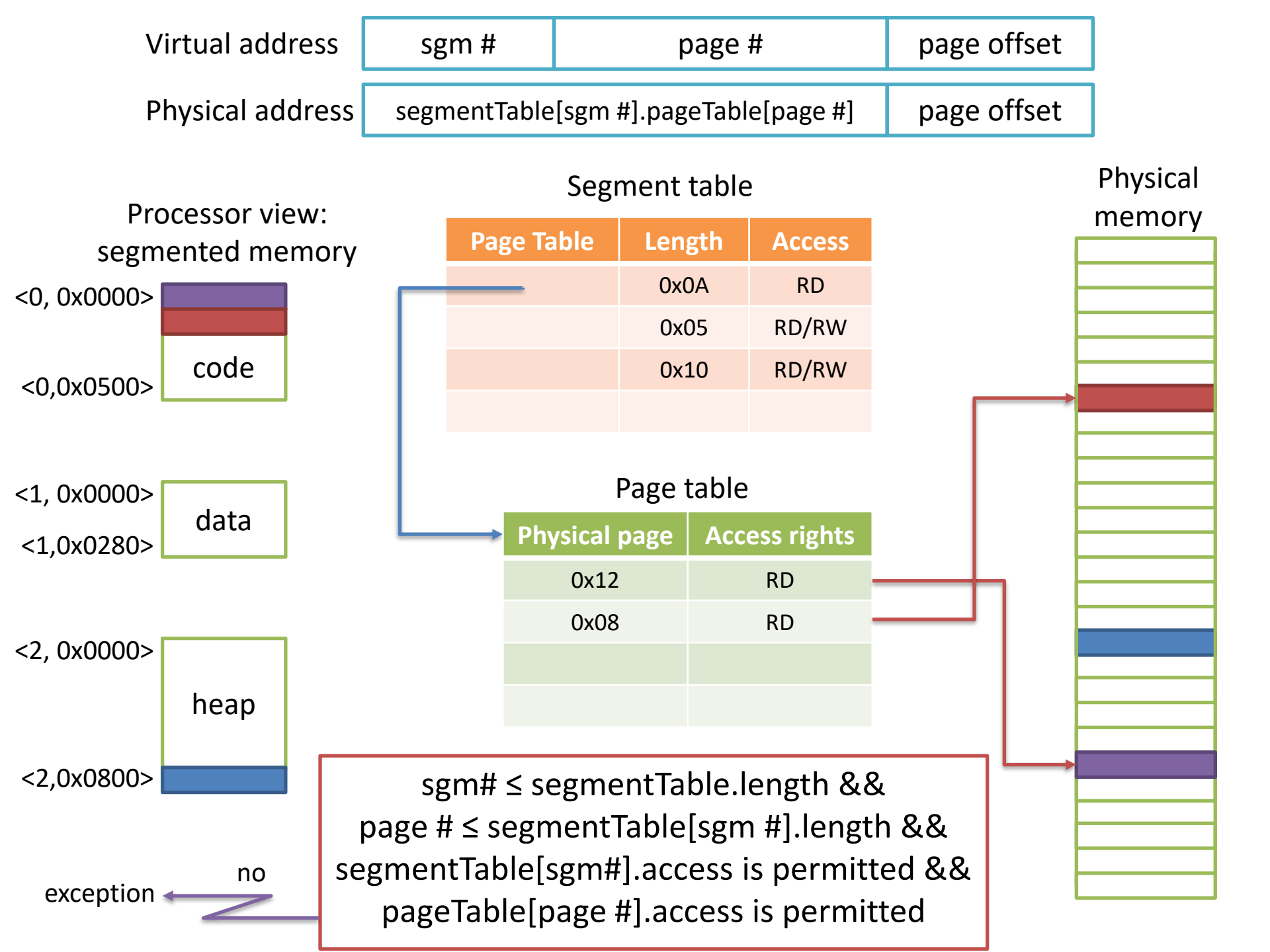
Multi-level Translation

- Tree of translation tables
  1. *Paged segmentation*
  2. *Multi-level page tables*
  3. *Multi-level paged segmentation*
- All these approaches: fixed size page as lowest level unit
  - Efficient memory allocation
  - Efficient disk transfers
  - Easier to build Translation Lookaside Buffers (TLBs)
  - Efficient reverse lookup (from physical -> virtual)
  - Page granularity for protection/sharing

Paged Segmentation

- Process memory is segmented
- The entry of the segment table is:
  - Pointer to page table
  - Page table length (# of pages in segment)
  - Access permissions
- The entry of the page table is:
  - Page frame
  - Access permissions
- Share/protection at either page or segment-level


Virtual address
segment #
page #
page offset
Physical address
SegmentTable[segment #].PageTable[page #]
page offset
Segment table
Page Table
Length
Access
0x0A
RD
0x05
RD/RW
0x10
RD/RW
Physical memory
Processor view: segmented memory
<0, 0x0000>
code
<0,0x0500>
<1, 0x0000>
data
<1,0x0280>
<2, 0x0000>
heap
<2,0x0800>
Page table
Physical page
Access rights
0x12
RD
0x08
RD

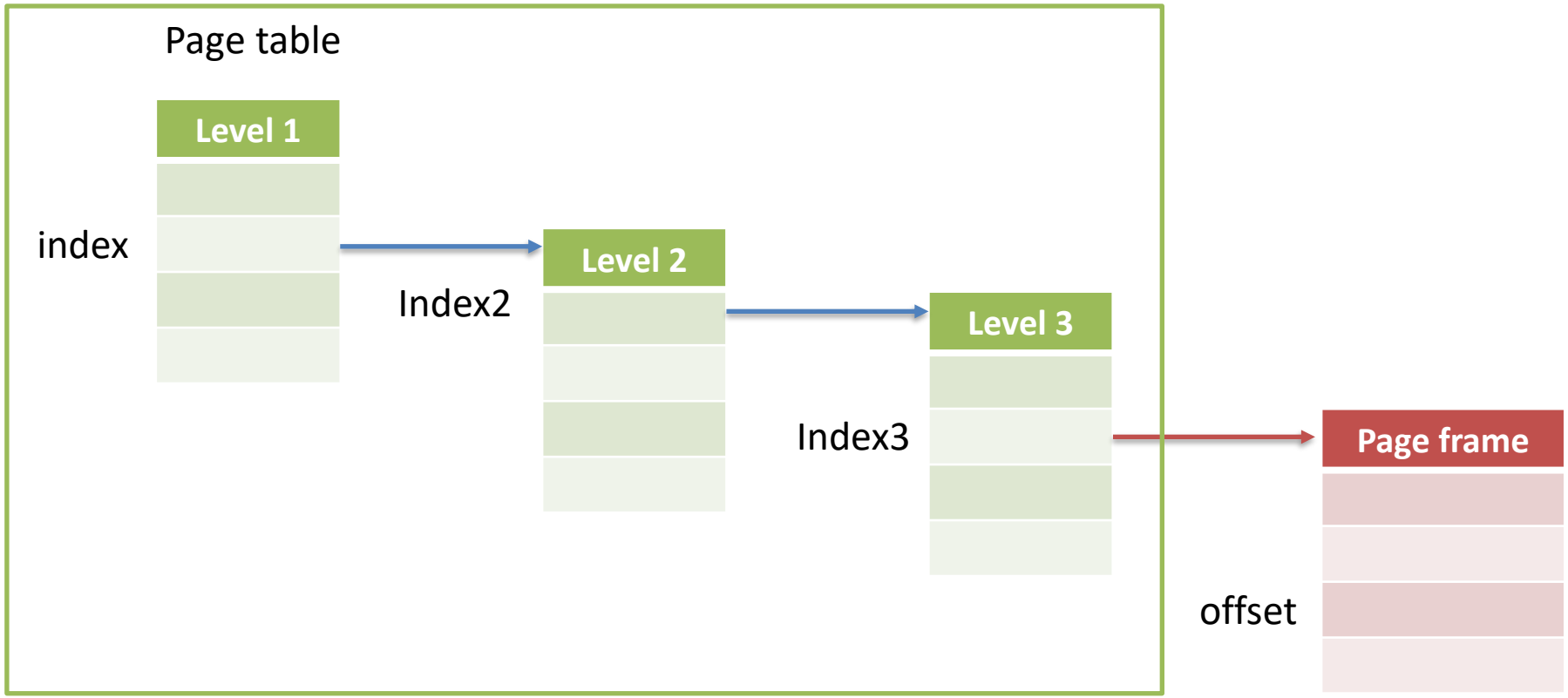

Virtual address
sgm #
page #
page offset
Physical address
segmentTable[sgm #].pageTable[page #]
page offset
Processor view:
segmented memory
<0, 0x0000>
code
<0,0x0500>
<1, 0x0000>
data
<1,0x0280>
<2, 0x0000>
heap
<2,0x0800>
Segment table
Page Table
Length
Access
0x0A
RD
0x05
RD/RW
0x10
RD/RW
Page table
Physical page
Access rights
0x12
RD
0x08
RD
Physical memory
sgm# ≤ segmentTable.length &&
page # ≤ segmentTable[sgm #].length &&
segmentTable[sgm#].access is permitted &&
pageTable[page #].access is permitted
no
exception

Question

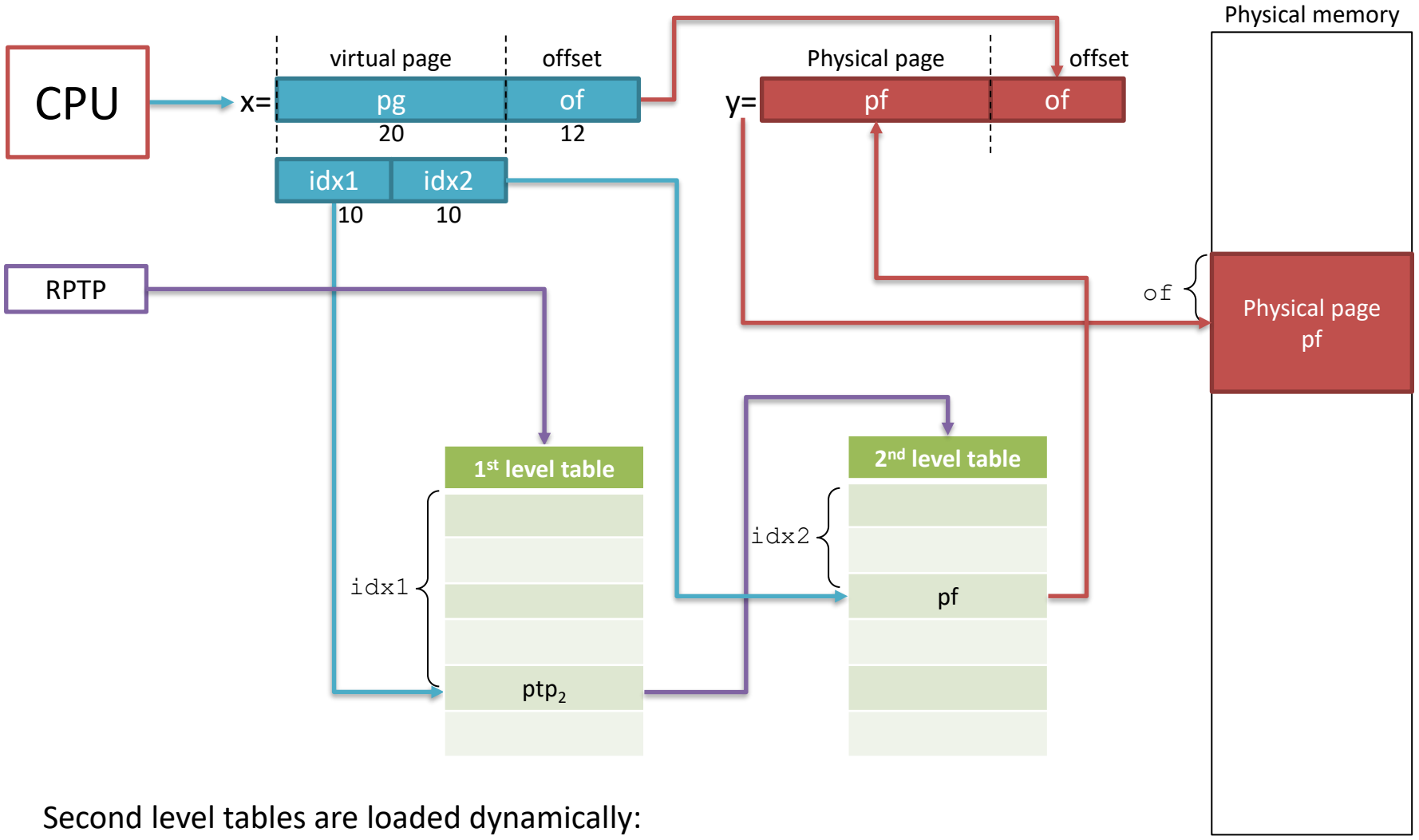
- With paged segmentation, what must be saved/restored across a process context switch?

Multilevel Paging

physical address=pageTable[index].pageTable[index2].pageTable[index3]|page offset



Two-level paging



Second level tables are loaded dynamically:

- reduce memory occupation

Comparing page table size

- Hypothesis:
  - 32 bit address;
  - logical and physical pages of 4K (→ 12 bit for the offset)
  - Page table entry of 4 byte
    - 3 bytes for block address
    - 1 byte for flags
- Page table size one single page level:
  - Number of table entries: 2^{20}
  - Space in bytes: $2^{20} * 4 = 2^{22}$ (4MB)
- Max physical memory size:
  - 3 bytes for the block address means 2^{24} pages → 64GB

Comparing page table size

- Page table size two-level paging (same hypothesis):
  - Of the 20 bits, 10 bits are for index1 and 10 bits for index2
  - We have 2^{10} second level tables
  - First and second level page table size is $2^{10} * 4 = 2^{12}$ (4K)
  - Best case: only 1 table in the second level → size = 8 K
  - Worst case: all tables in the second level → size = 4K + 4M
  - In general 4K + n*4K where n is the number of second level tables

- Max physical memory size:
  - 3 bytes for the block address means 2^{24} pages → 64GB

x86 Multilevel Paged Segmentation

- Global Descriptor Table (segment table)
  - Pointer to page table for each segment
  - Segment length
  - Segment access permissions
  - Context switch: change global descriptor table register (GDTR, pointer to global descriptor table)
- Multilevel page table
  - 4KB pages; each level of page table fits in one page
    - Only fill page table if needed
  - 32-bit: two level page table (per segment)
  - 64-bit: four level page table (per segment)

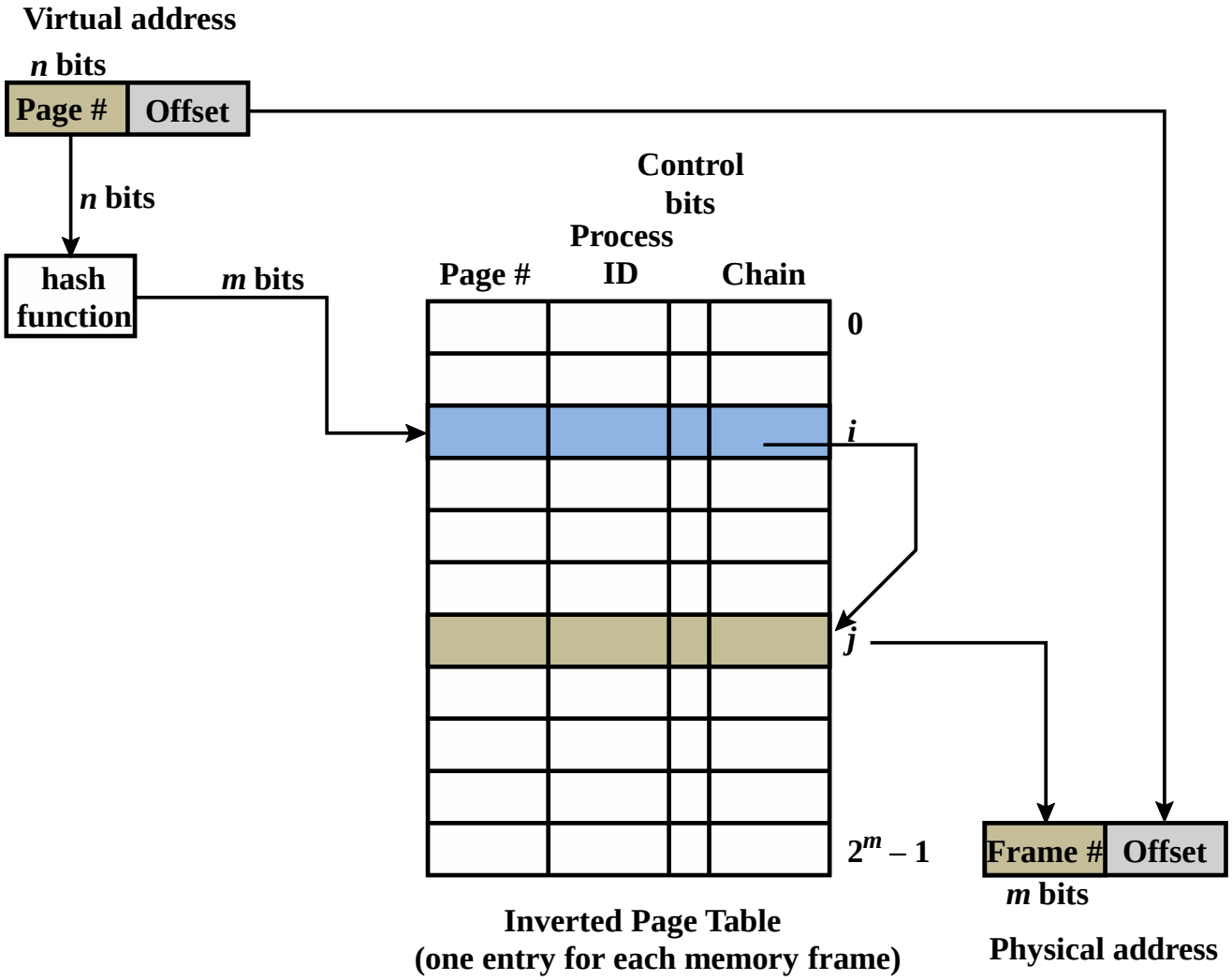
Multilevel Translation

- Pros:
  - Allocate/fill only as many page tables as used
  - Simple memory allocation
  - Share at segment or page level
- Cons:
  - Two or more lookups per memory reference

Portability

- Many operating systems keep their own memory translation data structures
  - List of memory objects (segments)
  - Virtual -> physical
  - Physical -> virtual (*core map*)
- Simplifies porting from different processors (x86, ARM, …) from 32 bit to 64 bit
- ***Inverted page table***
  - Hash from virtual page -> physical page
  - Space proportional to # of physical pages

Inverted Page Table



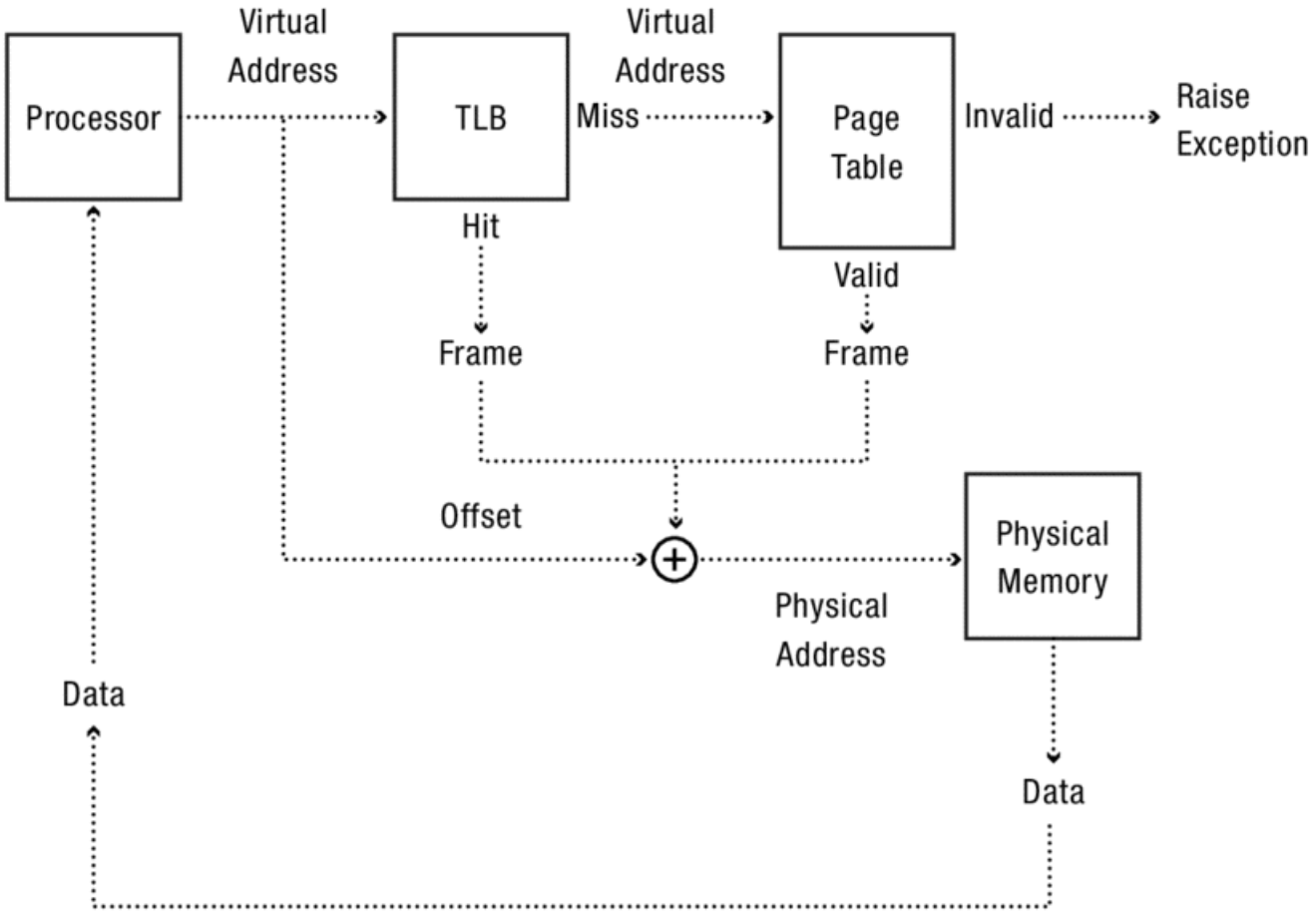
Do we need multi-level page tables?

- Use inverted page table in hardware instead of multilevel tree
  - IBM PowerPC
  - Hash virtual page # to inverted page table bucket
  - Location in Inverted Page Table => physical page frame
- Pros/cons?

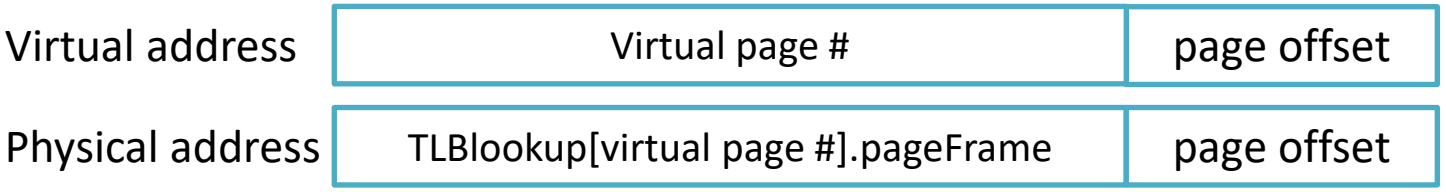
Efficient Address Translation

- Access to page tables has good ***locality***
- ***Translation Lookaside Buffer*** (TLB)
  - It is a cache of recent virtual page -> physical page translations
  - If cache hit, use translation available in the TLB entry
  - If cache miss, walk multi-level page table
- TLB cache is logically placed in the MMU
- Typical values for a TLB are:
  - Size: 16—512 entries; Block size: 1—2 page table entries
  - Hit time: 0.5—1 clock cycles; Miss Penalty: 10—100 clock cycles
  - Miss rate: 0.01%-1%
- We may have multiple TLB levels

Address Translation with TLB



$$Cost\ of\ Address\ Translation = TLB_{hit} + TLB_{miss} * CostFullTranslation$$



Processor view:
segmented memory

<0, 0x0000>

code

<0,0x0500>

<1, 0x0000>

data

<1,0x0280>

<2, 0x0000>

heap

<2,0x0800>

Translation Lookaside Buffer (TLB)

	virtual Page	Page Frame	Access
=?	0x0000	0x0A	RD
=?	0x40FF	0x05	RD/RW
=?	0x0001	0x10	RD/RW
=?			invalid

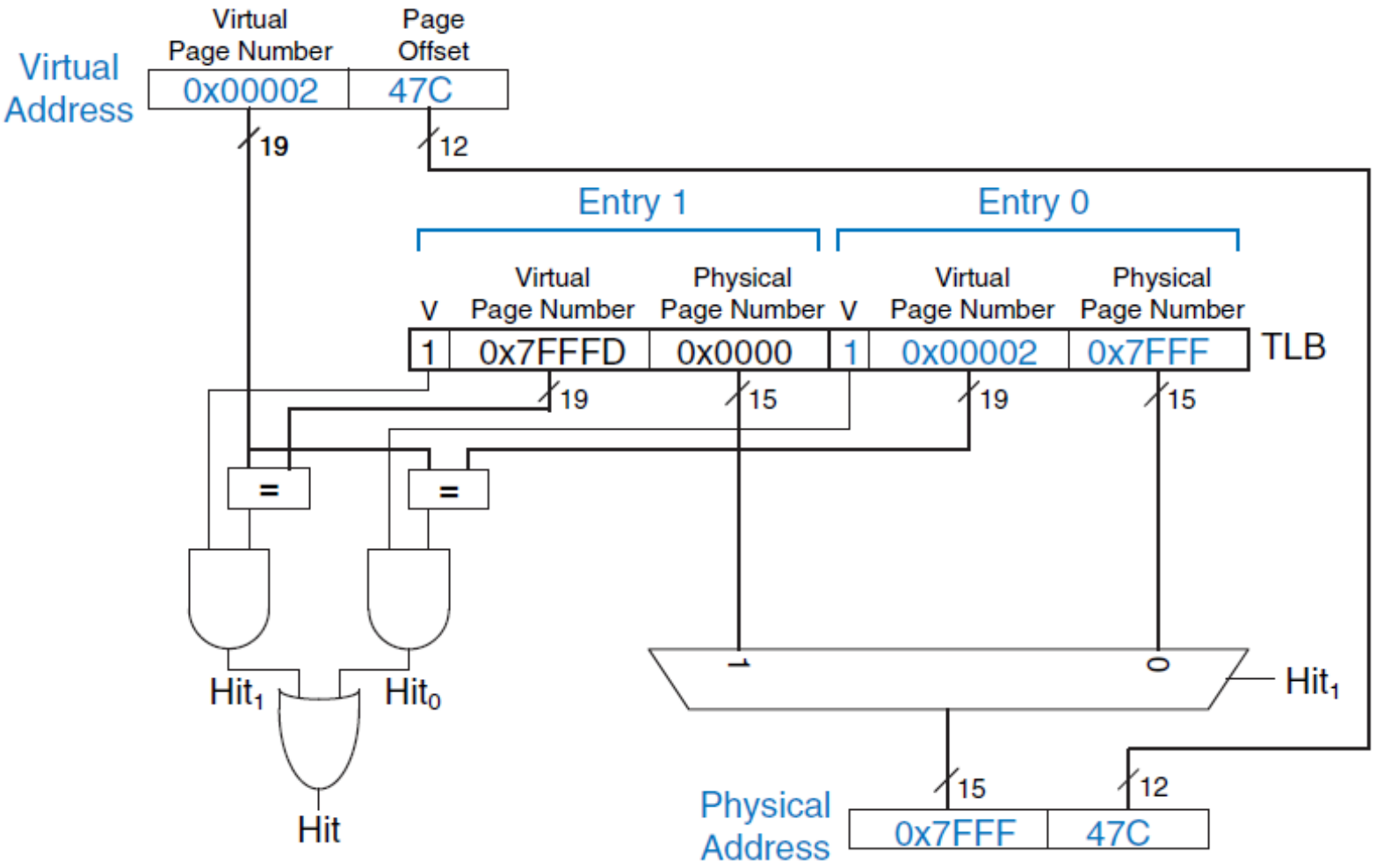
TLB contains Virtual page #?

no

do page table lookup

Physical
memory

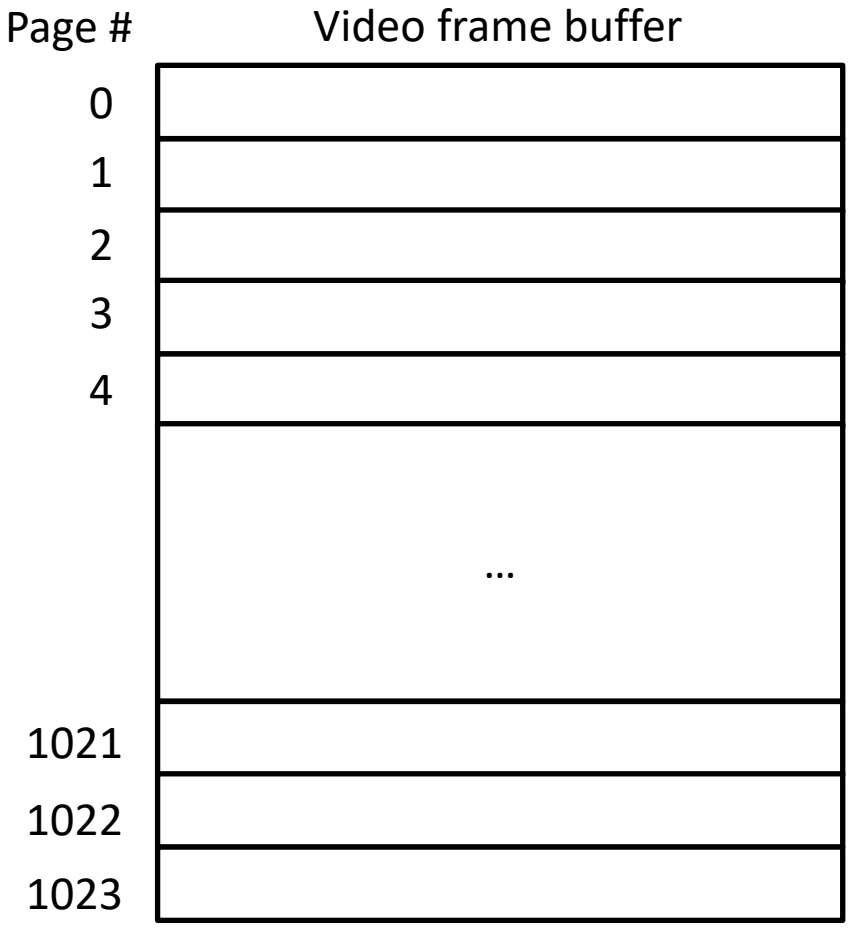
Two-entry TLB schema



Software Loaded TLB

- Do we need a page table at all?
  - MIPS processor architecture
  - If translation is in TLB, ok
  - If translation is not in TLB, trap to kernel
  - Kernel computes translation and loads TLB
  - Kernel can use whatever data structures it wants
- Pros/cons?

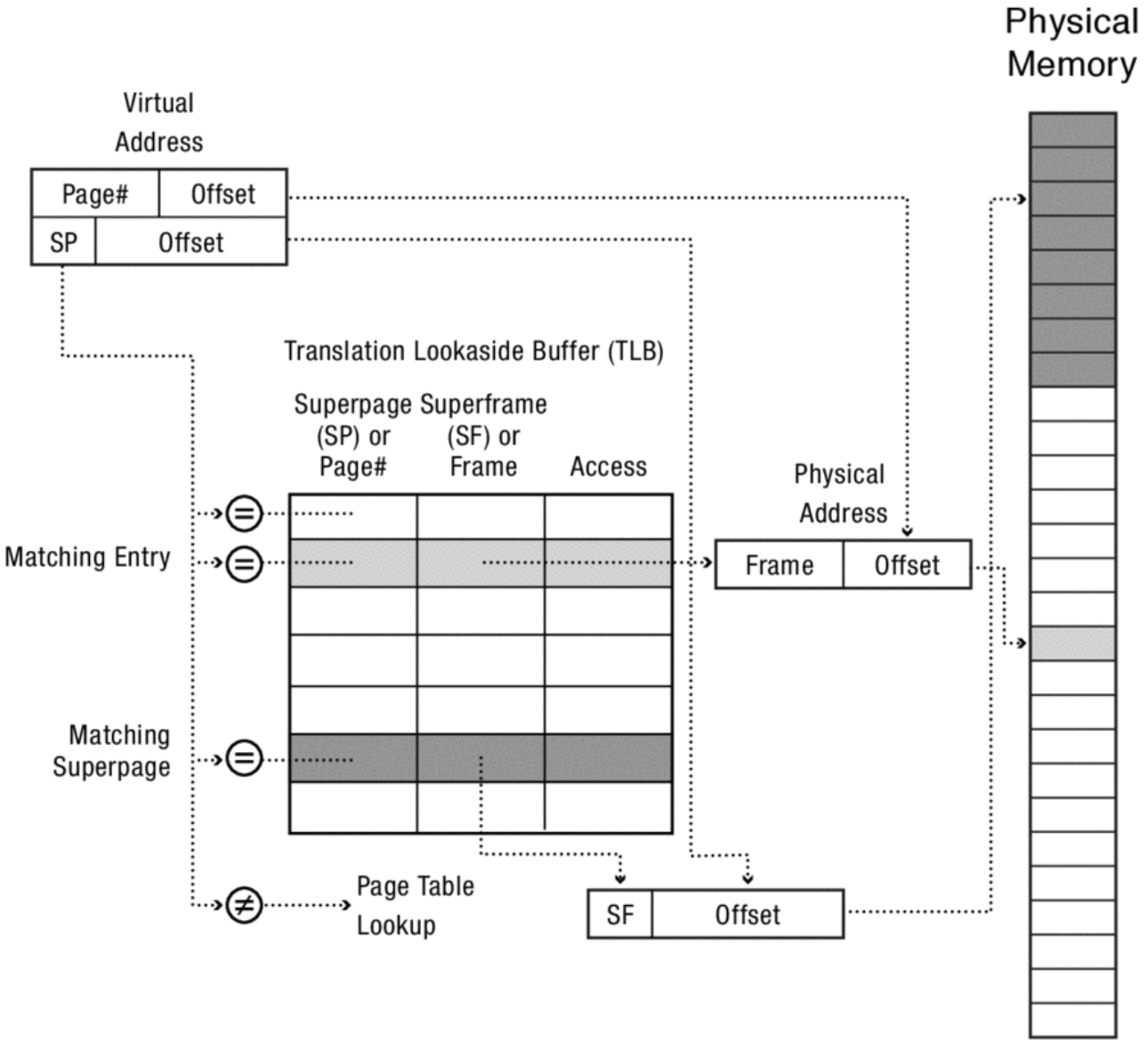
When Do TLBs Work/Not Work?



Superpages

- TLB entry can be
  - A page
  - A superpage: a set of contiguous pages
  - x86: superpage is set of pages in one page table
  - x86 TLB entries
    - 4KB
    - 2MB
    - 1GB

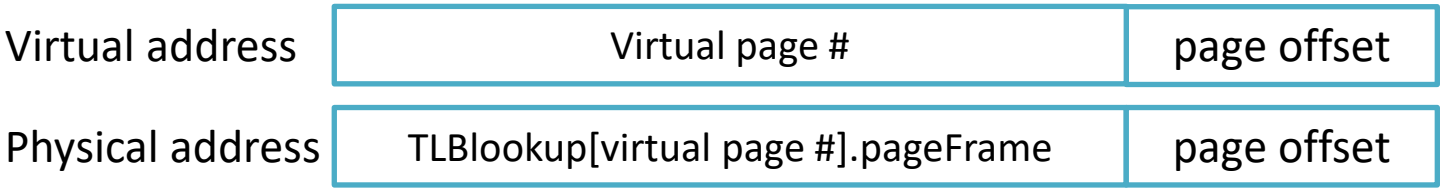
Superpages


Physical Memory
Virtual Address
Page#
Offset
SP
Offset
Translation Lookaside Buffer (TLB)
Superpage (SP) or Page#
Superframe (SF) or Frame
Access
Physical Address
Frame
Offset
Matching Entry
Matching Superpage
Page Table Lookup
SF
Offset

When Do TLBs Work/Not Work, part 2

- What happens on a context switch?
  - Reuse TLB?
  - Discard TLB?

- Motivates hardware tagged TLB
  - Each TLB entry has process ID
  - TLB hit only if process ID matches current process



Processor P view:
segmented memory

<0, 0x0000>

code

<0,0x0500>

<1, 0x0000>

data

<1,0x0280>

<2, 0x0000>

heap

<2,0x0800>

Translation Lookaside Buffer (TLB)

	PID	virtual Page	Page Frame	Access
=?	0	0x0000	0x0A	RD
=?	1	0x40FF	0x05	RD/RW
=?	1	0x0001	0x10	RD/RW
=?	0	0x003F	0x5B	RD

TLB contains Virtual page #
For the current process P?

no

do page table lookup

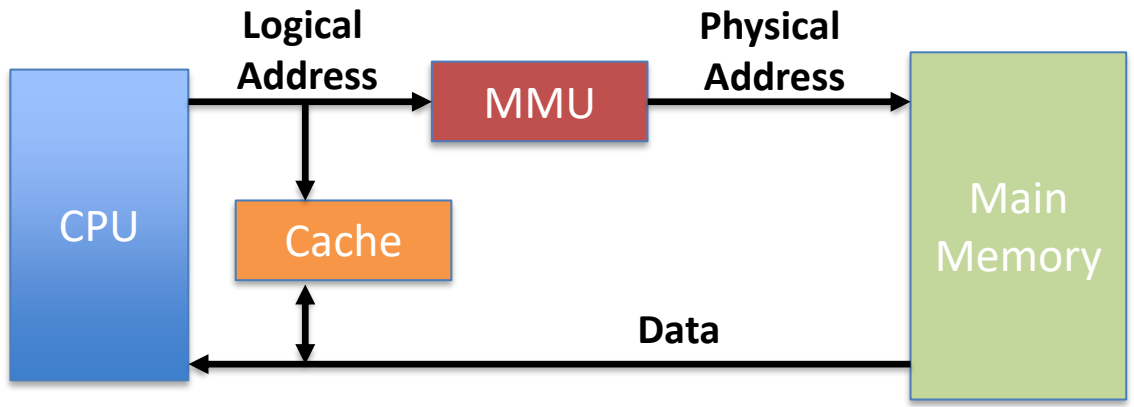
Physical
memory

When Do TLBs Work/Not Work, part 3

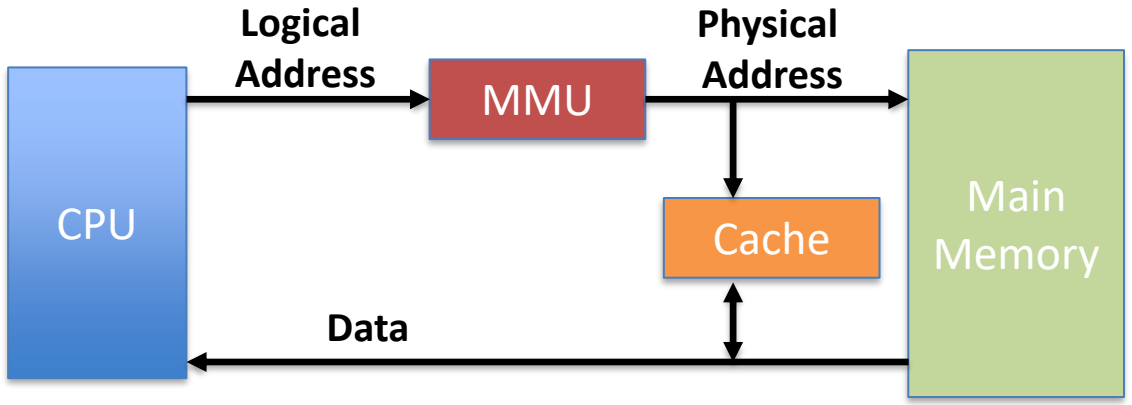
- What happens when the OS changes the permissions on a page?
  - For demand paging, copy on write, zero on reference, …
- TLB may contain old translation
  - OS must ask hardware to purge TLB entry
- On a multicore, ***TLB shootdown*** phenomenon
  - OS must ask each CPU to purge TLB entry

Virtual and Physical Cache

- Virtual cache faster because the cache can respond before the MMU performs an address translation
- But it requires to tag cache entries (same virtual address space for processes), thus more complexity and more space needed

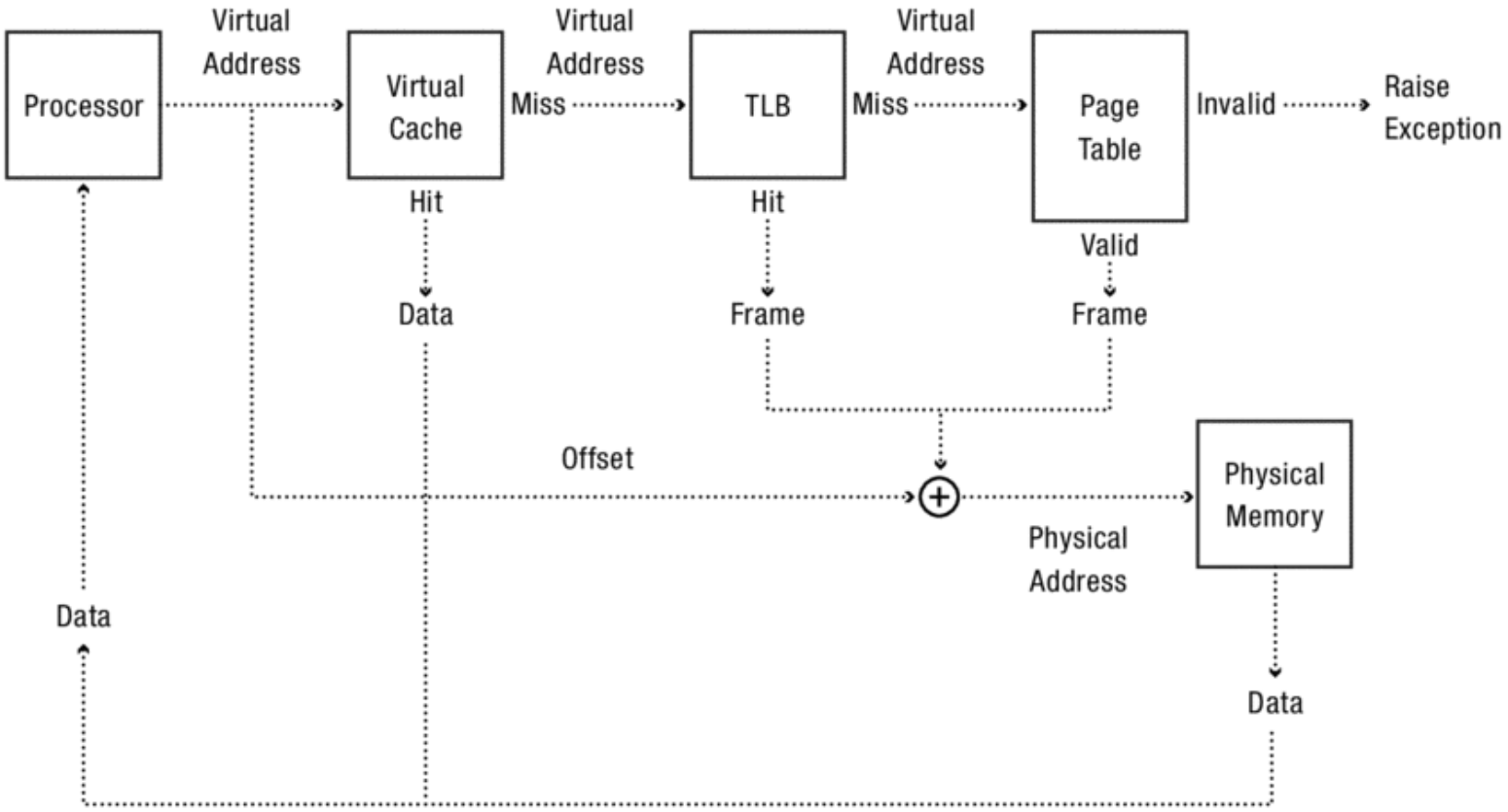


Virtual Cache

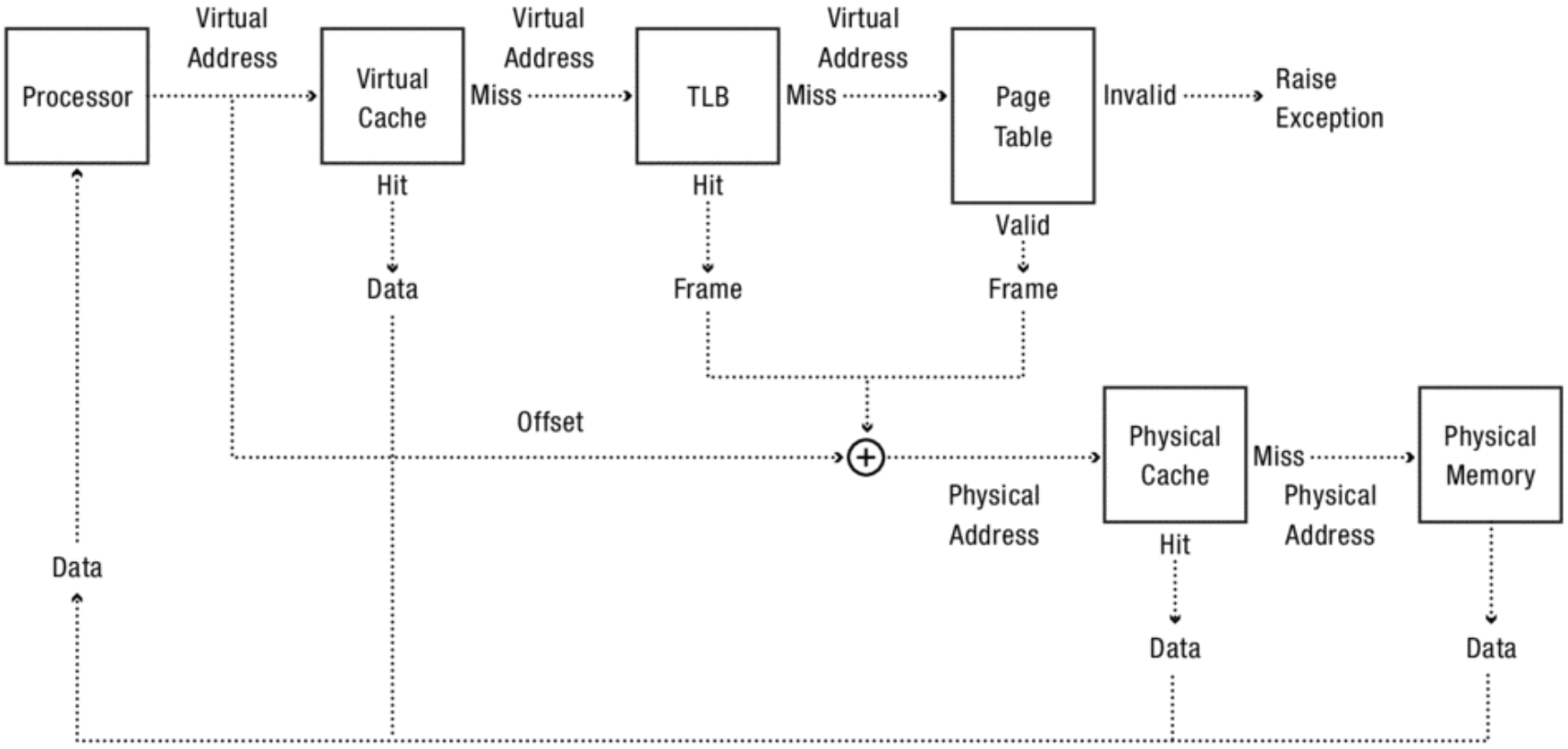


Physical Cache

Virtually addressed Cache



Physically addressed Cache



TLB and Cache Interaction